



Université Alioune Diop de Bambey

Master 2 Data Science et Génie Logiciel

Mini-projet JEE

Gestion des produits

API REST avec Spring Boot, Spring Data JPA et PostgreSQL

Présenté par : Papa Malick Ndiaye

Module : Java Entreprise Edition

Année universitaire 2025 - 2026

1. Objectif

Réaliser une API REST de gestion de produits (identifiant, nom, catégorie, prix, quantité) avec les opérations CRUD et quatre recherches : par catégorie, par nom, stock faible et plage de prix.

Le code source et la collection Postman sont sur GitHub. Une vidéo de démonstration a été envoyée à part.

Dépôt GitHub : https://github.com/pa-malick/Gestion_produit-JEE.git

2. Outils

Outil	Version	Rôle
Java (JDK)	23	Compilation et exécution
Spring Boot	4.1.0	Socle de l'application
Maven	3.9.9	Build et dépendances
PostgreSQL / pgAdmin 4	16 / 9.15	Base de données (port 5433)
Postman	12.23	Tests de l'API

Tableau 1 : outils utilisés

3. Architecture

Le projet suit une architecture en couches : Contrôleur → Service → Repository → PostgreSQL.

Couche	Fichier	Rôle
Entité	entity/Produit	Table produits
Repository	repository/ProduitRepository	Accès aux données
DTO	dto/ProduitRequest, ProduitResponse	Entrée validée, sortie
Mapper	mapper/ProduitMapper	Conversion DTO ↔ entité
Service	service/ProduitService + impl	Règles métier
Contrôleur	controller/ProduitController	Points d'entrée HTTP
Exceptions	exception/ (4 classes)	Réponses d'erreur uniformes

Tableau 2 : couches du projet

4. Entité et requêtes

L'entité Produit a un identifiant généré par la base, un nom unique et un prix en BigDecimal pour éviter les erreurs d'arrondi.

Méthode du repository	Recherche
findByCategorieIgnoreCase	par catégorie
findByNomContainingIgnoreCase	par morceau de nom
findByQuantiteLessThan	stock sous un seuil
findByPrixBetween	prix entre deux bornes
listerCategories (JPQL)	catégories distinctes
valeurStockParCategorie (JPQL)	valeur totale du stock

Tableau 3 : requêtes personnalisées

5. Validation et règles métier

- Nom et catégorie obligatoires, prix supérieur à 0, quantité positive (@Valid sur le DTO).
- Deux produits ne peuvent pas avoir le même nom.
- Pour la plage de prix, le minimum ne doit pas dépasser le maximum.
- La réponse ajoute valeurStock = prix × quantité.

6. Points d'entrée de l'API

Méthode	URL	Code
POST	/api/produits	201
GET	/api/produits	200
GET / PUT / DELETE	/api/produits/{id}	200 / 200 / 204
GET	/api/produits/categorie/{categorie}	200
GET	/api/produits/recherche?nom=	200
GET	/api/produits/stock-faible?seuil=	200
GET	/api/produits/prix?min=&max=	200
GET	/api/produits/categories	200
GET	/api/produits/valeur-stock?categorie=	200

Tableau 4 : points d'entrée

Erreur	Code
Produit inexistant	404
Nom déjà utilisé	409
Données ou paramètres invalides	400

Tableau 5 : gestion des erreurs (GlobalExceptionHandler)

7. Mise en route

Étape	Commande
Créer la base	createdb -U postgres -p 5433 produitsdb
Compiler	mvn clean package
Démarrer	java -jar target/gestion-produits-0.0.1-SNAPSHOT.jar

Tableau 6 : commandes

Hibernate crée la table, puis data.sql insère 8 produits. L'API écoute sur http://localhost:8080.

8. Tests avec Postman

La collection contient 15 requêtes en trois dossiers : CRUD, recherches et cas d'erreur.

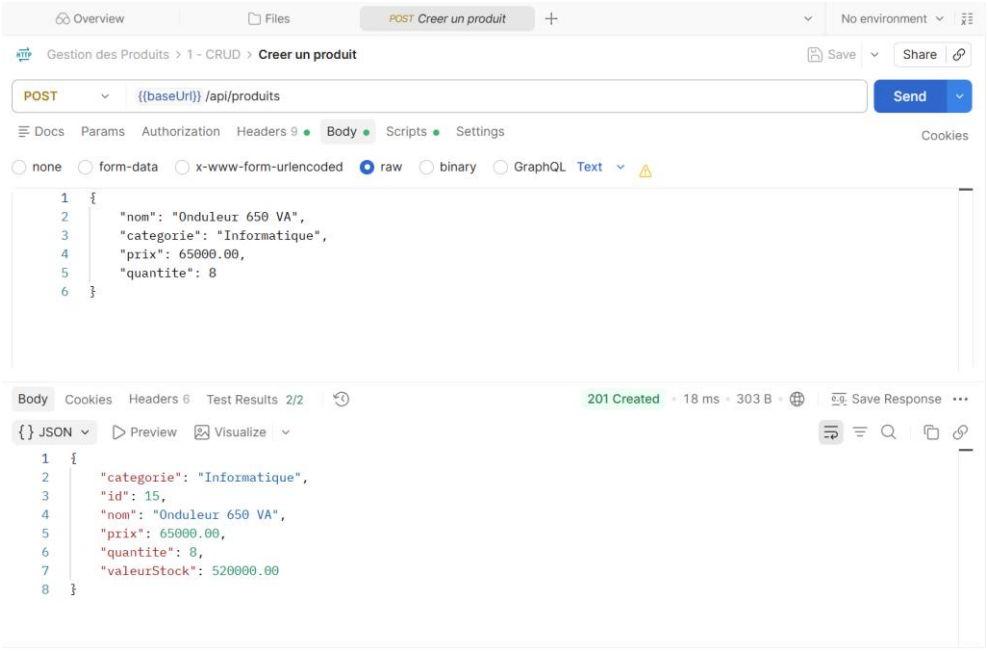


Figure 1 : création d'un produit (201)

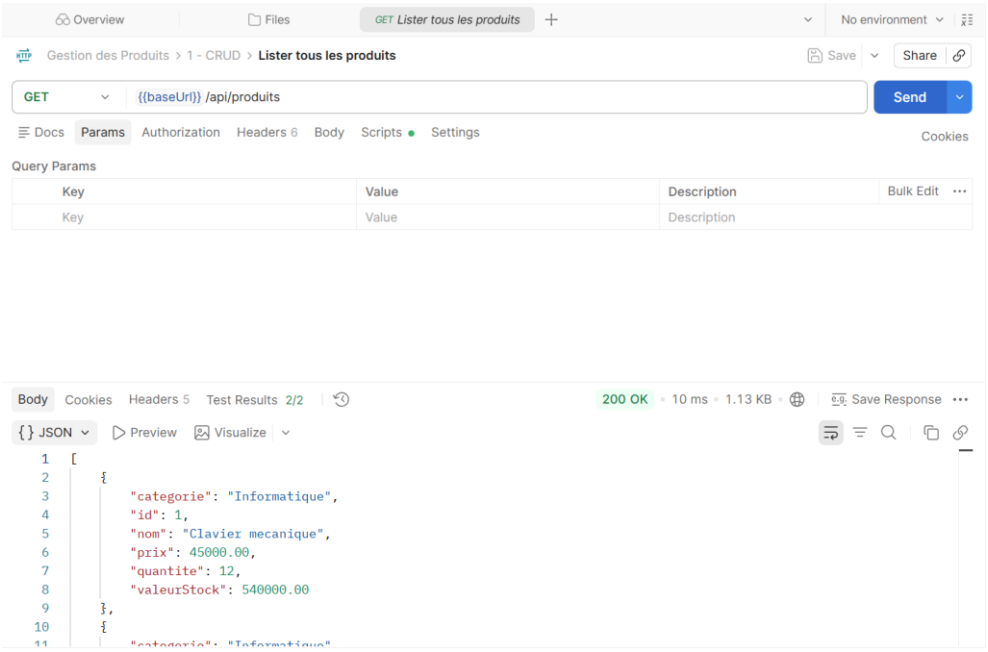


Figure 2 : liste des produits (200)

La consultation, la modification et la suppression renvoient 200, 200 et 204.

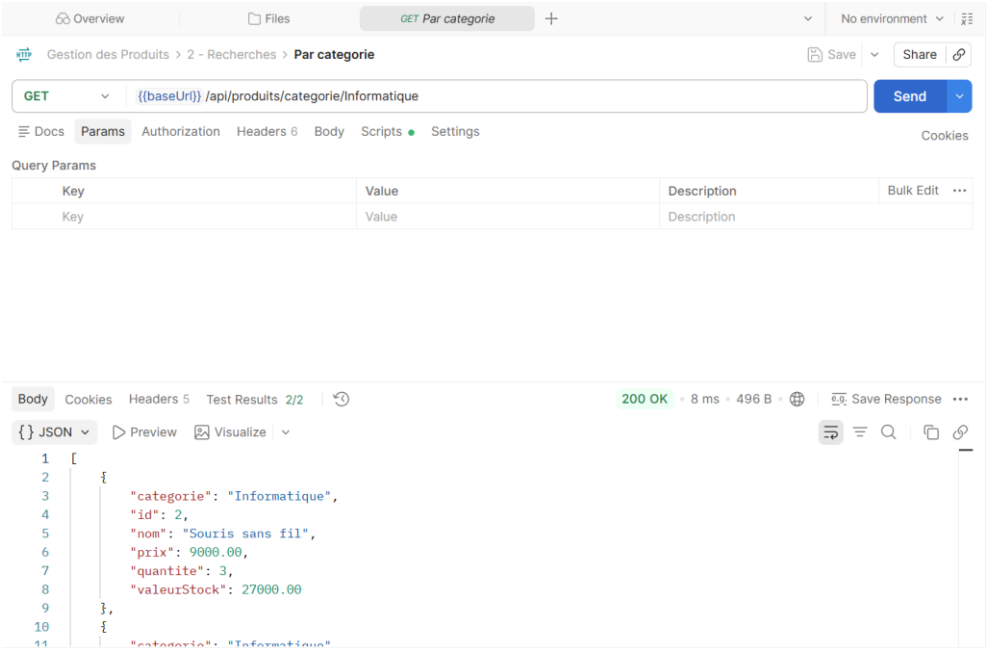


Figure 3 : produits de la catégorie Informatique

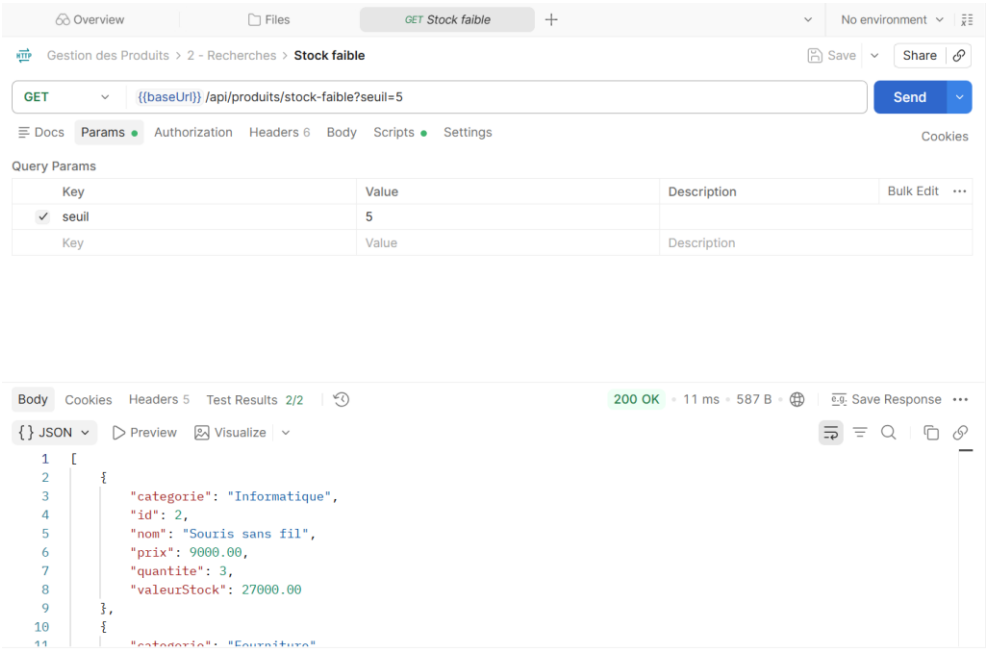


Figure 4 : produits en stock faible (seuil 5)

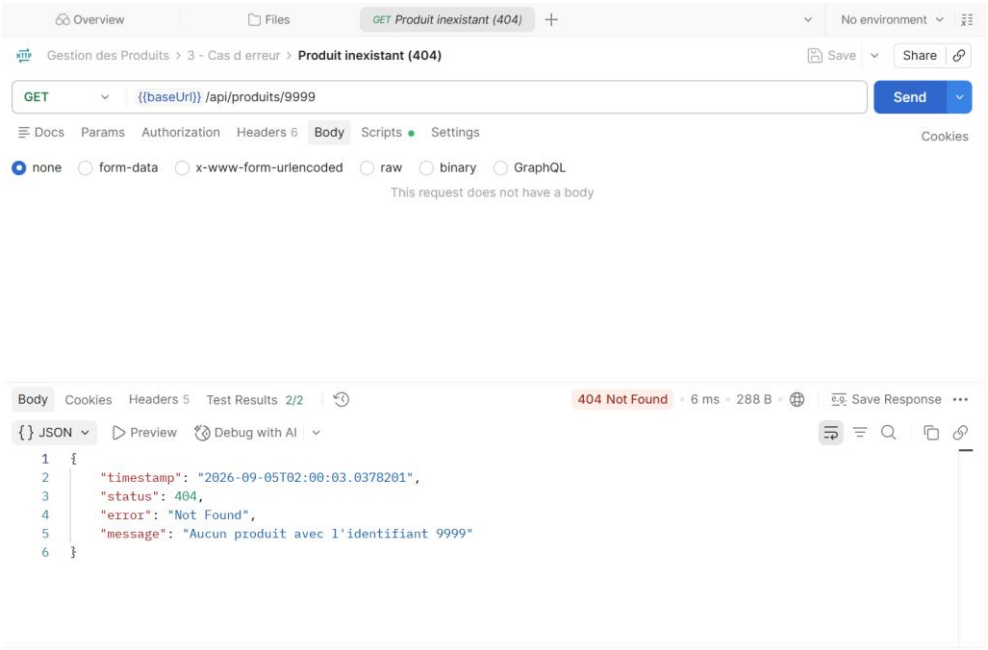


Figure 5 : produit inexistant (404)

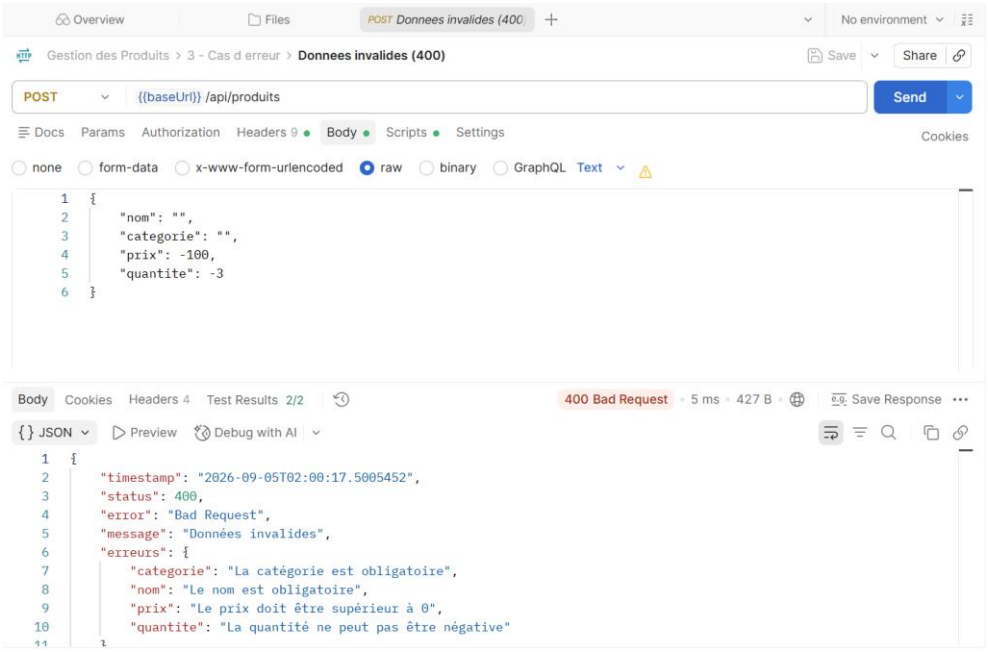


Figure 6 : données invalides (400)

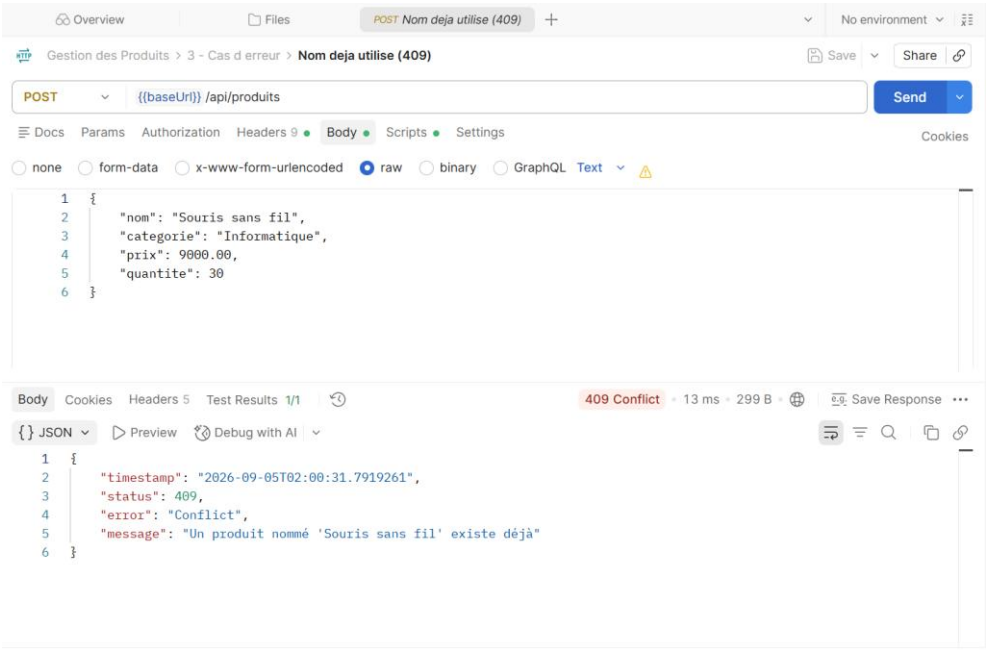


Figure 7 : nom déjà utilisé (409)

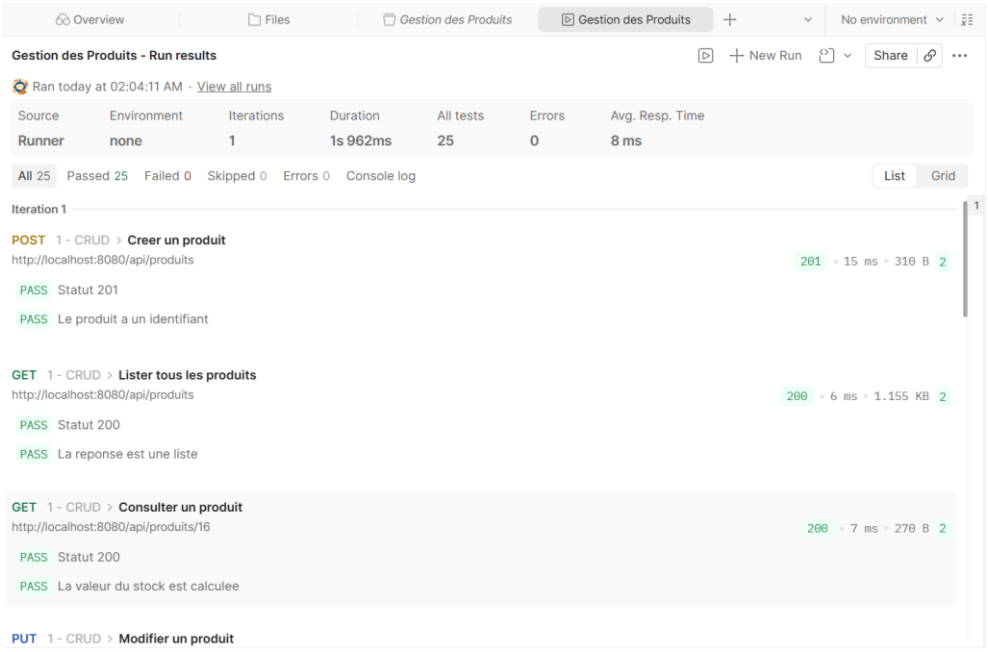


Figure 8 : exécution complète : 25 assertions réussies

9. Vérification dans pgAdmin 4

Le produit créé dans Postman apparaît bien dans la table produits, puis disparaît après la suppression.

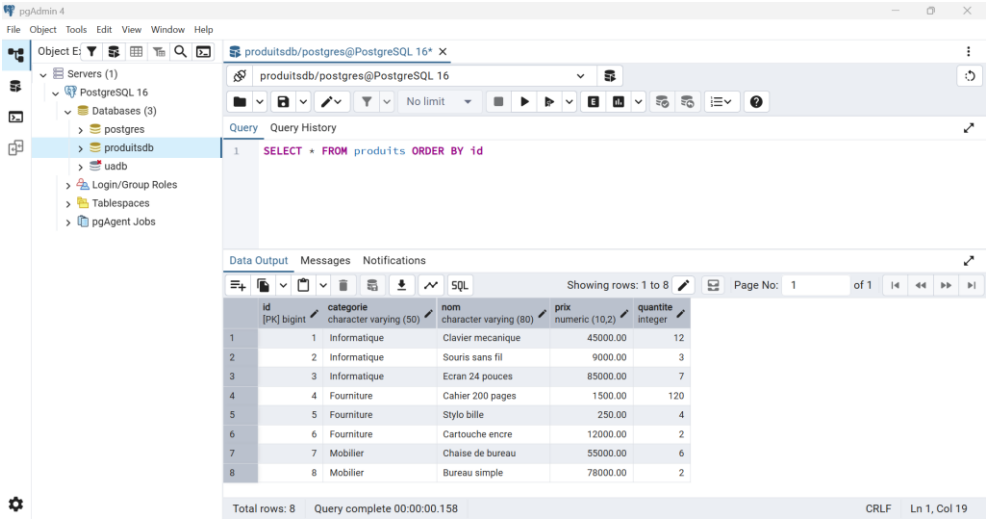


Figure 9 : table produits au départ : 8 produits

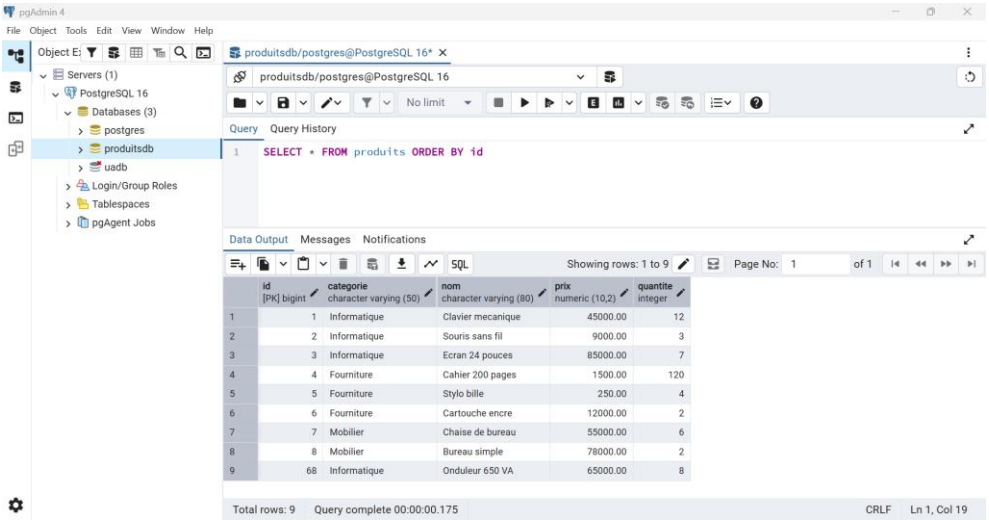


Figure 10 : après la création dans Postman : 9 produits

Les en-têtes de colonnes montrent la structure de la table : clé primaire bigint, nom en varchar(80), catégorie en varchar(50), prix en numeric(10,2) et quantité en integer.

10. Difficultés rencontrées

- PostgreSQL écoute sur le port 5433 et non 5432 : l'URL de connexion a été adaptée.
- data.sql s'exécutait avant la création de la table : réglé avec spring.jpa.defer-datasource-initialization.
- Une variable produitId vide dans l'environnement Postman masquait celle de la collection : elle a été retirée.

11. Conclusion

Tous les points demandés sont réalisés et testés : 15 requêtes, 25 assertions réussies, et les données sont vérifiées dans pgAdmin. Le découpage en couches rend le code simple à lire et à faire évoluer.